



UNIVERSIDAD CATÓLICA BOLIVIANA "SAN PABLO"

DEPARTAMENTO DE CIENCIAS DE LA TECNOLOGÍA E INNOVACIÓN

INGENIERÍA MECATRÓNICA

Sistema Automático de Clasificación de Tomates mediante Visión Artificial y ESP32

INGENIERÍA MECATRÓNICA



Materia: Sistemas Embebidos II

Estudiante(s): Susann Baldiviezo C.

Alejandro Bejarano R.

Florencia Frigerio A.

Benjamín Soruco G.

Docente: Msc. Ing. Alan Cornejo Q.

Semestre: 1/2026

Bolivia, 8 de junio de 2026

Índice

1. Descripción técnica-conceptual del proyecto a realizar	1
2. Propósito del proyecto	2
3. Alcance del proyecto	3
3.1. Lo que el sistema SÍ contempla	3
3.2. Lo que el sistema NO contempla	3
4. Supuestos del proyecto	4
5. Requerimientos	4
5.1. Requerimientos funcionales	4
5.2. Requerimientos no funcionales	5
5.3. Requerimientos de hardware	5
6. Entregables principales del proyecto	7
7. Fundamentos teóricos (conocimientos básicos)	7
8. Arquitectura de comunicaciones y protocolos	8
8.1. Topología de red	8
8.2. Tópicos MQTT	9
8.3. Secuencia de mensajes	9
9. Diseño del firmware embebido	10
9.1. Pinout del nodo ESP32 Motor	10
9.2. Pinout del nodo ESP32 Consola	10
9.3. Consumo eléctrico de la consola portátil	10
9.4. Concurrencia con FreeRTOS: disparo no bloqueante de servomotores	11
9.5. Máquinas de estados finitos (FSM) actualizadas	12
10. Caracterización y gráficas de respuesta	14
10.1. Señal PWM del motor	14
10.2. Cronograma del disparo de servomotores	15
10.3. Presupuesto de latencia extremo a extremo	16
11. Aplicación real: producción de tomate en Tarija	17
Referencias Bibliográficas	19

Índice de figuras

1.	Diagrama en bloques del sistema actualizado con servomotores MG996R. La Raspberry Pi actúa como concentrador de mensajería (broker MQTT) y punto de acceso WiFi; los nodos se comunican de forma indirecta a través de tópicos[cite: 189, 190, 191].	2
2.	Diagrama de secuencia MQTT para el arranque (comando del operario) y para un fruto rechazado. El broker desacopla a los nodos y replica cada mensaje a los suscriptores[cite: 268, 269].	9
3.	FSM del nodo ESP32 Motor actualizada con servomotores MG996R. El estado DISPARO SERVOS se ejecuta en una tarea FreeRTOS separada (no bloqueante, duración total 350 ms)[cite: 296, 297].	13
4.	FSM de pantallas de la consola OLED. La transición forzada a ALERTA se dispara con tres rechazos consecutivos[cite: 299, 300].	14
5.	Señal PWM a 10 kHz en GPIO25 con ciclo de trabajo de 150/255 (58.8 %). Período de 100 μ s[cite: 303, 304].	15
6.	Cuantización del PWM: el <i>duty</i> de 8 bits se mapea linealmente a la tensión media en el motor. En rojo, el punto de arranque (<i>duty</i> =150)[cite: 304, 305].	15
7.	Cronograma del disparo de servomotores gestionado por la tarea FreeRTOS dedicada. Incluye rampas de giro y estabilización[cite: 309, 310].	16
8.	Presupuesto de latencia extremo a extremo (valores típicos/de especificación). La inferencia domina; la cadena MQTT + reacción embebida es despreciable frente a ella[cite: 313, 314].	17

Índice de tablas

1.	Componentes de hardware del sistema (actualizado con servomotores). . .	6
2.	Software, frameworks y bibliotecas.	6
3.	Entregables del proyecto.	7
4.	Parámetros de la red local del sistema.	8
5.	Tópicos MQTT, publicadores, suscriptores y cargas útiles (payloads). . . .	9
6.	Pinout del ESP32 Motor (driver BTS7960 + servomotores MG996R). . . .	10
7.	Pinout del ESP32 Consola (OLED I ² C, botones).	10
8.	Presupuesto de consumo eléctrico de la consola portátil.	11

1. Descripción técnica-conceptual del proyecto a realizar

El proyecto consiste en el diseño e implementación de un sistema automático de clasificación de tomates sobre una banda transportadora, que separa físicamente los frutos en buen estado de aquellos en estado de descomposición[cite: 169]. La detección se realiza por visión artificial con OpenCV Gonzalez y Woods, 2018; OpenCV Development Team, 2024, mientras que toda la cadena de control, actuación, comunicación y monitoreo se resuelve con **sistemas embebidos basados en el microcontrolador ESP32** Espressif Systems, 2023, que constituyen el núcleo de este documento[cite: 170]. El sistema se organiza en cuatro capas: (i) *adquisición*, con cámaras USB sobre la banda[cite: 171]; (ii) *procesamiento*, en una PC que ejecuta el algoritmo de visión[cite: 172]; (iii) *mensajería*, en una Raspberry Pi que actúa como *broker* MQTT Light, 2017 y punto de acceso WiFi [cite: 173]; y (iv) *actuación y monitoreo*, con dos ESP32 independientes: uno controla el motor de la banda y dos servomotores MG996R para desvío, y el otro es una consola portátil inalámbrica para el operario[cite: 174]. La PC emite un veredicto binario por cada fruto (*aceptada* o *rechazada*) y lo publica en la red [cite: 175]; los nodos embebidos reaccionan a ese mensaje en tiempo real[cite: 176].

El principal desafío técnico no está en la visión, sino en la **coordinación distribuida en tiempo real** entre nodos heterogéneos sin un cableado punto a punto: tres equipos (PC y dos ESP32) deben compartir estado de forma confiable, con baja latencia y de manera robusta ante desconexiones, sobre una red local que además debe funcionar *sin acceso a Internet*[cite: 177]. La propuesta de valor es ofrecer una solución de control de calidad poscosecha automatizada, portátil y de bajo costo, reproducible con componentes disponibles en el mercado local, orientada a pequeños productores (Sección 11)[cite: 178]. En la Figura 1 se presenta el diagrama en bloques del sistema[cite: 179]. Se observa la separación en capas y, en particular, que la Raspberry Pi concentra la mensajería: ningún nodo se comunica directamente con otro, sino que todos publican y se suscriben a través del *broker*, lo que desacopla a los dispositivos entre sí[cite: 180].

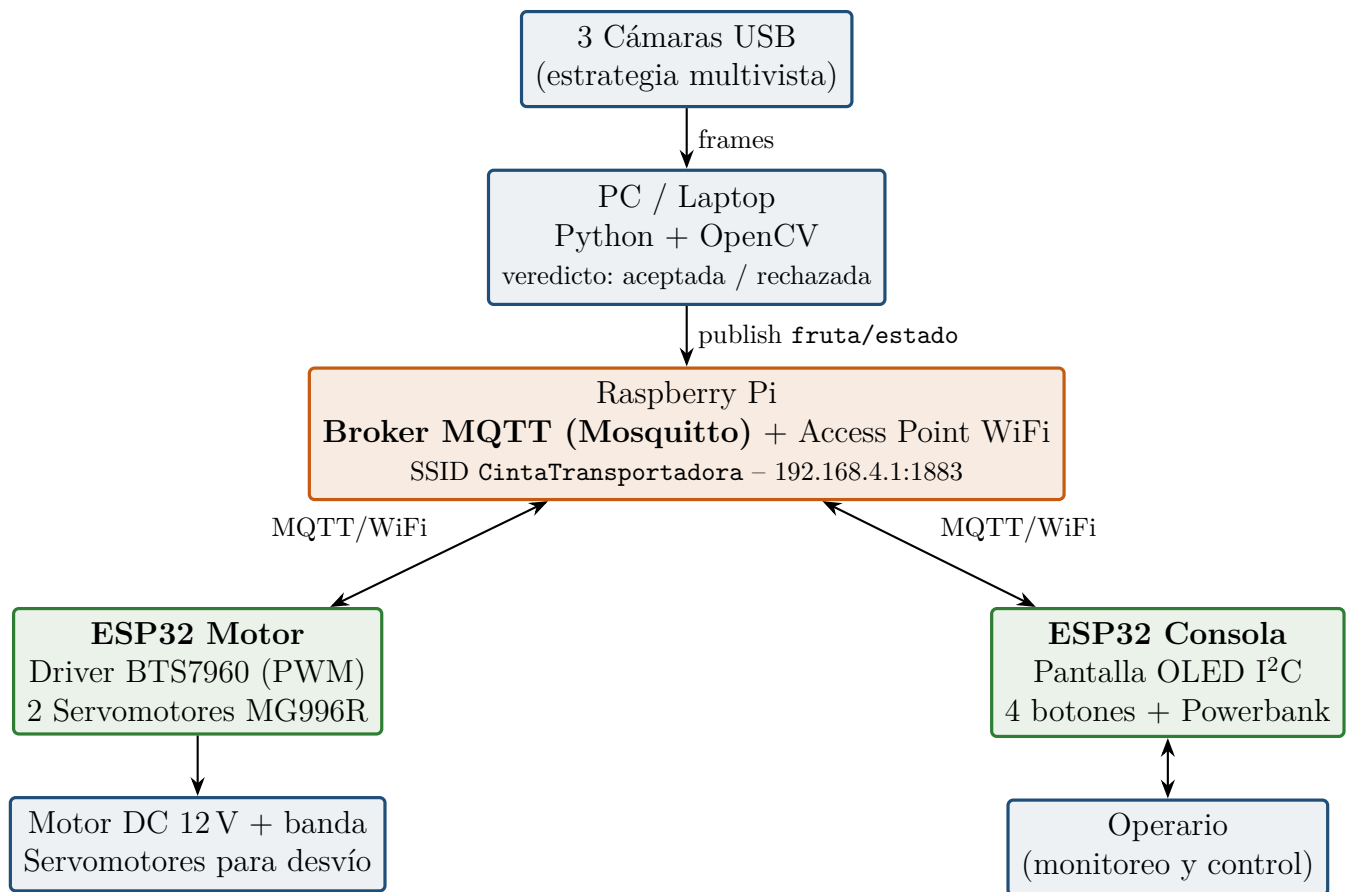


Figura 1: Diagrama en bloques del sistema actualizado con servomotores MG996R. La Raspberry Pi actúa como concentrador de mensajería (broker MQTT) y punto de acceso WiFi; los nodos se comunican de forma indirecta a través de tópicos[cite: 189, 190, 191].

2. Propósito del proyecto

El propósito es automatizar la clasificación poscosecha de tomate por estado de sanidad (sano vs. deteriorado), integrando visión artificial con un sistema embebido distribuido que controle los actuadores y permita la supervisión del operario en tiempo real[cite: 191]. En concreto se busca:

- Reducir la intervención humana en una tarea repetitiva y con alto margen de error visual, separando automáticamente el fruto deteriorado[cite: 192].
- Disminuir las pérdidas económicas asociadas a la comercialización de fruto en mal estado y al contagio de la podredumbre dentro del lote[cite: 193].
- Demostrar la viabilidad técnica de coordinar varios nodos embebidos ESP32 con una PC mediante un protocolo de mensajería liviano (MQTT) sobre una red WiFi local autónoma[cite: 194].

- Aplicar de forma integrada los contenidos de *Sistemas Embebidos II*: GPIO, PWM, I²C, conectividad WiFi, multitarea con FreeRTOS Barry, 2016 y control de actuadores de potencia[cite: 195].

Desde lo académico, el énfasis está en el **firmware embebido y la arquitectura de comunicaciones**; la visión artificial se trata como un módulo proveedor de veredictos cuya implementación interna es intercambiable[cite: 196, 197].

3. Alcance del proyecto

El proyecto abarca el diseño, la construcción y la validación de un prototipo funcional[cite: 198]. A continuación se delimita el alcance.

3.1. Lo que el sistema SÍ contempla

- Firmware embebido para dos ESP32 desarrollado sobre ESP-IDF v5.x Espressif Systems, 2024: control de motor por PWM, disparo de servomotores, lectura de botones y manejo de display OLED[cite: 199].
- Control de la banda transportadora mediante driver de potencia BTS7960 (puente H) Infineon Technologies, 2004[cite: 200].
- Expulsión física del fruto rechazado mediante dos servomotores MG996R acoplados mecánicamente con precisión angular de ± 5 [cite: 201].
- Arquitectura de comunicaciones completa: red WiFi local con la Raspberry Pi como Access Point y *broker* MQTT, definición de tópicos, calidad de servicio y secuencia de mensajes[cite: 202].
- Consola portátil (segunda ESP32) con pantalla OLED I²C (128x64 px), cuatro botones, alimentada por *powerbank* 12000 mAh, para monitoreo y control inalámbrico[cite: 203].
- Clasificación binaria: *buen estado* / *mal estado*, bajo iluminación controlada de laboratorio[cite: 204].

3.2. Lo que el sistema NO contempla

El presente proyecto no incluye:

- Clasificación por tamaño, peso o grado de madurez (solo estado de descomposición)[cite: 205].
- Operación en líneas industriales de alta velocidad o con varios tipos de fruta en simultáneo[cite: 206].

- Seguridad/cifrado de la red MQTT (se asume red local privada y aislada)[cite: 207].
- Integración con sistemas ERP o trazabilidad industrial[cite: 208].

4. Supuestos del proyecto

Para el desarrollo del proyecto se supone que:

- **Iluminación controlada:** el prototipo opera bajo luz artificial constante que minimiza variaciones que afecten el análisis de color y textura[cite: 208].
- **Frutos individualizados:** los tomates se colocan sobre la banda separados entre sí, de modo que el sistema los analice de forma independiente[cite: 209].
- **Velocidad de banda calibrada:** la velocidad es fija (14 segundos entre estaciones) y permite a la visión capturar y procesar cada fruto a tiempo (Sección 10)[cite: 210].
- **Red WiFi local disponible:** la Raspberry Pi genera el Access Point CintaTransportadora; no se requiere Internet[cite: 211].
- **Tierra común:** todos los GND (ESP32, BTS7960, fuente 12V, servomotores) están unidos; sin ello el sistema se comporta de forma errática[cite: 212, 213].
- **Ambiente de laboratorio:** el prototipo se diseña para taller académico, no para entornos con polvo, humedad o vibración industrial[cite: 213].

5. Requerimientos

5.1. Requerimientos funcionales

1. Sistema principal de clasificación:

- a) RF-01: el sistema debe capturar imágenes de cada fruto y emitir un veredicto binario[cite: 214].
- b) RF-02: la ESP32 Motor debe arrancar/detener la banda según el comando del operario[cite: 215].
- c) RF-03: ante un veredicto **rechazada**, la ESP32 Motor debe disparar los servomotores y expulsar el fruto[cite: 216].
- d) RF-04: el disparo de los servomotores no debe bloquear la atención de nuevos mensajes MQTT[cite: 217].

2. Consola portátil del operario:

- a) RF-05: la consola debe mostrar contadores de aceptadas/rechazadas y el estado del motor con latencia menor a 1s[cite: 218].

- b) RF-06: el operario debe poder iniciar y detener todo el proceso desde la consola, sin acceder a la PC[cite: 219].
- c) RF-07: la consola debe indicar mediante pantalla OLED un volumen elevado de rechazos consecutivos[cite: 220].
- d) RF-08: la consola debe indicar la pérdida de conexión con el sistema central[cite: 221].
- e) RF-09: la consola debe funcionar de forma autónoma (*powerbank*), sin cables[cite: 222].

5.2. Requerimientos no funcionales

1. RNF-01: el tiempo de procesamiento y reacción por fruto no debe superar los 2 s[cite: 223].
2. RNF-02: el sistema debe operar de forma continua al menos 30 min sin fallos[cite: 224].
3. RNF-03: el firmware debe estar estructurado de forma modular y documentado[cite: 225].
4. RNF-04: el prototipo debe ser reproducible con componentes de bajo costo del mercado local[cite: 226].
5. RNF-05: el sistema debe operar sin acceso a Internet (red local autónoma)[cite: 227].

5.3. Requerimientos de hardware

En la Tabla 1 se listan los componentes de hardware[cite: 228]. En la Tabla 2 se listan las herramientas y bibliotecas de software con sus versiones[cite: 229].

Tabla 1: Componentes de hardware del sistema (actualizado con servomotores).

ID	Componente	Cant.
H-01	ESP32 WROOM-32 (nodo Motor)	1
H-02	ESP32 WROOM-32 (nodo Consola)	1
H-03	Raspberry Pi (broker MQTT + Access Point)	1
H-04	Driver BTS7960 / IBT-2 (puente H)	1
H-05	Motor DC 12 V + banda transportadora	1
H-06	Servomotor MG996R	2
H-07	Fuente 12 V (motor + servos)	1
H-08	3 cámaras USB	3
H-09	PC / Laptop (procesamiento OpenCV)	1
H-10	Pantalla OLED I ² C 128x64 px (SSD1306)	1
H-11	4 pulsadores (botones)	4
H-12	Powerbank 5 V 12000 mAh (alimentación consola)	1
H-13	Iluminación LED controlada	1

Tabla 2: Software, frameworks y bibliotecas.

Componente	Rol	Versión
ESP-IDF	Framework de desarrollo ESP32	v5.0+
FreeRTOS	RTOS (multitarea) incluido en ESP-IDF	—
<code>espressif/mqtt</code>	Cliente MQTT del ESP32	IDF v5.x
Mosquitto	Broker MQTT (Raspberry Pi)	2.x
NetworkManager (<code>nmcli</code>)	Access Point WiFi en la RPi	—
Python	Módulo de visión y simulador	3.10+
OpenCV	Visión por computadora	4.8+
paho-mqtt	Cliente MQTT en Python	1.6+

6. Entregables principales del proyecto

Tabla 3: Entregables del proyecto.

Entregable	Descripción
Prototipo físico	Banda con 3 cámaras, servomotores y nodos ESP32 integrados y funcionando[cite: 232].
Firmware ESP32 Motor	Código embebido modular (motor, servomotores, MQTT, WiFi) sobre ESP-IDF[cite: 233].
Firmware ESP32 Consola	Código de la consola portátil (OLED, botones, MQTT)[cite: 234].
Configuración Raspberry Pi	Broker Mosquitto, Access Point y servicio <code>systemd</code> [cite: 235].
Esquemas y pinouts	Conexiones de hardware y diagramas (Secciones 9–10)[cite: 236].
Memoria del trabajo final	El presente documento Universidad Católica Boliviana "San Pablo", 2026[cite: 236].
Demostración en vivo	Sistema funcionando con tomates reales ante el docente[cite: 237].

7. Fundamentos teóricos (conocimientos básicos)

ESP32 y FreeRTOS. El ESP32 es un SoC con doble núcleo Xtensa LX6, WiFi 802.11 b/g/n y Bluetooth integrados Espressif Systems, 2023[cite: 238]. El framework ESP-IDF lo programa en C sobre **FreeRTOS**, un kernel de tiempo real que permite ejecutar varias *tareas* concurrentes con planificación por prioridades Barry, 2016[cite: 239]. Esto es clave en el proyecto: una operación que tarda (girar los servomotores 350 ms) se delega a una tarea propia para no bloquear la atención de la red[cite: 240].

GPIO y PWM (periférico LEDC). Los pines de propósito general (GPIO) leen botones[cite: 241]. Para regular la velocidad del motor se usa **PWM** (modulación por ancho de pulso): el periférico LEDC del ESP32 genera una onda cuadrada de frecuencia fija (10 kHz) cuyo *ciclo de trabajo* (*duty*, 0–255 en resolución de 8 bits) fija la tensión media aplicada al motor a través del BTS7960[cite: 242]. Los servomotores se controlan con PWM a 50 Hz con ancho de pulso variable (1.0–2.0 ms para 0°–180°)[cite: 243].

Puente H BTS7960. Es un medio puente de alta corriente; dos integrados forman un puente H completo (módulo IBT-2) capaz de manejar el motor DC de 12 V, con entradas RPWM/LPWM para sentido/velocidad y R_EN/L_EN para habilitación Infineon Technologies, 2004[cite: 244, 245].

Servomotor MG996R. Servomotor de torque alto (11 kg·cm a 6 V), velocidad 0.2 s/60° y rango 0°–180°. Se controla mediante PWM a 50 Hz; el ancho de pulso (1.0–2.0 ms) determina la posición angular[cite: 246]. Dos servos acoplados mecánicamente permiten expulsar frutos rechazados con precisión y retornar a posición neutral[cite: 247].

I²C y pantalla OLED. La pantalla OLED 128x64 px (controlador SSD1306) se comunica por I2C a 100 kHz (dirección típica 0x3C), con solo dos pines (SDA y SCL)[cite: 248]. Ofrece mejor visibilidad que LCD, bajo consumo de corriente (20 mA típico) y funcionamiento con batería[cite: 249].

WiFi (modos STA y AP). Cada ESP32 opera como estación (STA) y se asocia al Access Point creado por la Raspberry Pi (modo AP)[cite: 250]. Así se forma una red local cerrada, sin router ni Internet IEEE, 2021[cite: 251].

MQTT. Protocolo de mensajería *publicación/suscripción* liviano sobre TCP, ideal para IoT Banks y Gupta, 2014[cite: 252]. Un *broker* (Mosquitto) recibe mensajes publicados en *tópicos* y los reenvía a los clientes suscritos[cite: 253]. La calidad de servicio QoS 1 garantiza “al menos una entrega”[cite: 254]. Este desacople publicador/suscriptor es lo que permite agregar o quitar nodos sin reconfigurar a los demás[cite: 255].

8. Arquitectura de comunicaciones y protocolos

Esta es la columna vertebral del sistema[cite: 256]. La PC y los dos ESP32 no se comunican entre sí directamente: todos lo hacen a través del *broker* MQTT alojado en la Raspberry Pi[cite: 257].

8.1. Topología de red

La Raspberry Pi crea un Access Point WiFi WPA2 (canal 7, banda 2.4 GHz) mediante `nmcli`, con IP fija 192.168.4.1, y los clientes obtienen IP por DHCP[cite: 258]. En la Tabla 4 se resumen los parámetros de red.

Tabla 4: Parámetros de la red local del sistema.

Parámetro	Valor
SSID	CintaTransportadora
Seguridad	WPA2-PSK (clave cinta2026)
Banda / canal	2.4 GHz / 7
IP del broker (RPi, AP)	192.168.4.1
Puerto MQTT	1883 (TCP)
Asignación de IP a clientes	DHCP (192.168.4.x)
Autenticación del broker	anónima (<code>allow_anonymous true</code>)

8.2. Tópicos MQTT

La Tabla 5 define el contrato de communication: cada tópico, quién publica, quién se suscribe y los valores válidos[cite: 259]. Todos los mensajes se intercambian con QoS 1[cite: 260].

Tabla 5: Tópicos MQTT, publicadores, suscriptores y cargas útiles (payloads).

Tópico	Publica	Se suscribe	Valores
fruta/estado	PC (OpenCV)	ESP32 Motor, Consola	aceptada, rechazada
operario/comando	ESP32 Consola	ESP32 Motor	iniciar, detener
sistema/motor	ESP32 Motor	ESP32 Consola	activo, detenido, esp32_online
sistema/servos	ESP32 Motor	ESP32 Consola	expulsado

8.3. Secuencia de mensajes

La Figura 2 muestra la secuencia para dos eventos típicos: el arranque por comando del operario y la llegada de un fruto rechazado[cite: 260]. Se aprecia cómo el *broker* replica cada mensaje a todos los suscriptores, de modo que la consola y el motor mantienen estado coherente[cite: 261].

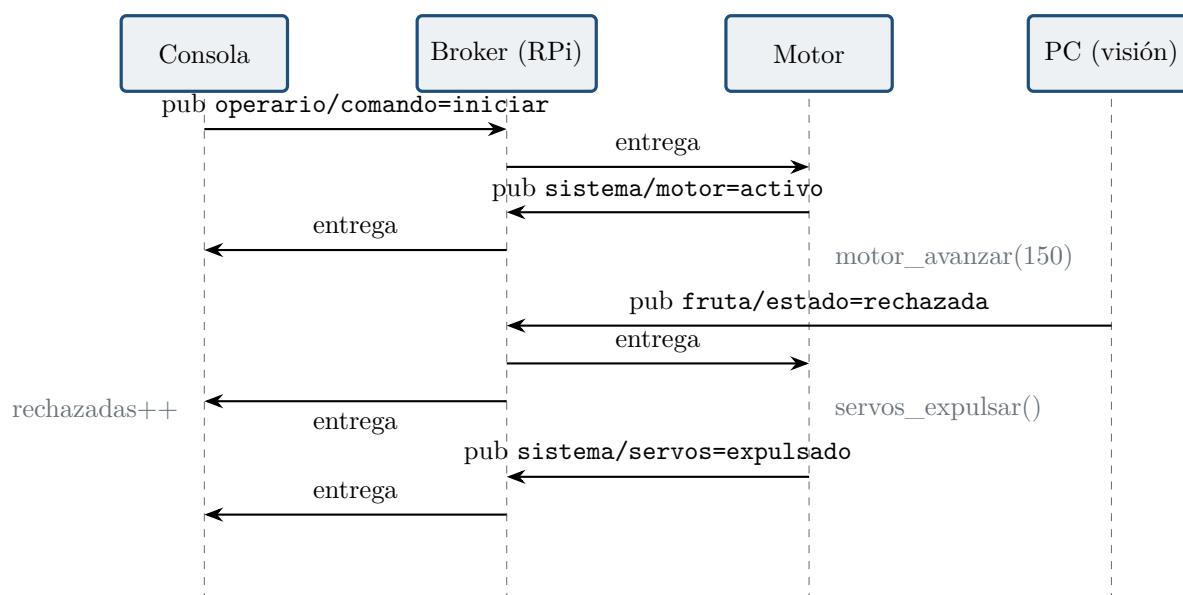


Figura 2: Diagrama de secuencia MQTT para el arranque (comando del operario) y para un fruto rechazado. El broker desacopla a los nodos y replica cada mensaje a los suscriptores[cite: 268, 269].

9. Diseño del firmware embebido

El firmware se desarrolla en C sobre ESP-IDF con un diseño modular[cite: 269]. El nodo Motor separa responsabilidades en `config.h` (pines, red, tópicos), `motor`, `servos`, `mqtt_handler` y `wifi_manager` [cite: 270]; el nodo Consola concentra OLED, botones y MQTT con tareas FreeRTOS dedicadas[cite: 271].

9.1. Pinout del nodo ESP32 Motor

La Tabla 6 detalla la asignación de pines del nodo Motor, derivada de `config.h` y del cableado del BTS7960 y los servomotores[cite: 272].

Tabla 6: Pinout del ESP32 Motor (driver BTS7960 + servomotores MG996R).

GPIO	Señal	Destino	Dir.
25	RPWM (PWM adelante)	BTS7960 RPWM	salida
26	LPWM (PWM atrás)	BTS7960 LPWM	salida
32	R_EN (habilita der.)	BTS7960 R_EN	salida
33	L_EN (habilita izq.)	BTS7960 L_EN	salida
4	PWM Servo 1	Servomotor 1 (50 Hz)	salida
5	PWM Servo 2	Servomotor 2 (50 Hz)	salida
5V/GND	Alimentación lógica	BTS7960, servos	—

9.2. Pinout del nodo ESP32 Consola

La Tabla 7 detalla los pines del nodo Consola (OLED I²C, botones con *pull-up* interno)[cite: 273, 274].

Tabla 7: Pinout del ESP32 Consola (OLED I²C, botones).

GPIO	Señal	Destino	Dir.
21	SDA (I ² C)	OLED (SSD1306 @ 0x3C)	E/S
22	SCL (I ² C)	OLED (SSD1306 @ 0x3C)	salida
14	Botón INICIAR	a GND (pull-up int.)	entrada
27	Botón DETENER	a GND (pull-up int.)	entrada
26	Botón PANTALLA	a GND (pull-up int.)	entrada
25	Botón RESET	a GND (pull-up int.)	entrada

9.3. Consumo eléctrico de la consola portátil

La consola está alimentada íntegramente desde una powerbank 12000mAh (capacidad nominal 12Wh @ 5V)[cite: 274, 275]. En la Tabla 8 se desglosa el consumo de cada componente y el tiempo de autonomía estimado[cite: 275].

Tabla 8: Presupuesto de consumo eléctrico de la consola portátil.

Componente	Tipo	Voltaje	Corriente	Potencia	Duty	Notas
ESP32 (WiFi + core)	MCU	3.3 V	80 mA	0.26 W	100 %	WiFi activo
OLED 128x64	Display	3.3 V	20 mA	0.07 W	100 %	Brillo normal
Botones (pull-ups)	Digital	3.3 V	5 mA	0.02 W	100 %	4 resistencias 10 k
Total ESP32			105 mA	0.35 W		
DC-DC Regulador	Boost 5V3.3V	5 V	63 mA	0.32 W	91 %	Eficiencia típica
Consumo Total		5 V	168 mA	0.84 W		
<i>Autonomía con powerbank 12000 mAh @ 5V nominal: Tiempo = 12000 mAh / 168 mA $\approx$ 71 horas (continuo) Operación típica ($\sim$8 horas/turno): Múltiples turnos sin recarga</i>						

En la práctica, el consumo puede variar: modo profundo (light sleep) reduce a $\sim$ 10 mA; actividad de botones es esporádica[cite: 277, 278]. La powerbank ofrece **autonomía de 2–3 días de operación típica** (8 horas/día)[cite: 278].

9.4. Concurrencia con FreeRTOS: disparo no bloqueante de servomotores

El patrón embebido más relevante del proyecto resuelve RF-04[cite: 279]. Cuando llega **fruta/estado=rechazada**, el *callback* de MQTT **no** gira los servomotores directamente (eso lo bloquearía 350 ms), sino que *notifica* a una tarea dedicada mediante `xTaskNotifyGive` [cite: 280]; la tarea mueve los servos, espera con `vTaskDelay` y publica la confirmación, todo sin congelar el hilo de red[cite: 281]. El Código 1 muestra esta tarea[cite: 282].

```

1 static void tarea_servos_expulsion(void *arg) {
2     while (1) {
3         // Bloquea aquí hasta recibir xTaskNotifyGive() desde el callback MQTT
4         ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
5         ESP_LOGI(TAG, ">>> Disparando servomotores a 180 grados");
6         servo_set_angle(1, 180);    // Servo 1 abre (180°)
7         servo_set_angle(2, 180);    // Servo 2 abre (180°)
8
9         vTaskDelay(pdMS_TO_TICKS(300)); // Mantener 300 ms
10
11        servo_set_angle(1, 90);      // Servo 1 retorna a neutral (90°)
12        servo_set_angle(2, 90);      // Servo 2 retorna a neutral (90°)
13
14        vTaskDelay(pdMS_TO_TICKS(50)); // Estabilización
15
16        ESP_LOGI(TAG, ">>> Servomotores retornados a neutral");
17        if (cliente_mqtt)
18            esp_mqtt_client_publish(cliente_mqtt, TOPIC_SERVOS, "expulsado", 0, 1, 0);
19    }
20 }
```

Listing 1: Tarea FreeRTOS de los servomotores: espera una notificación, gira a 180°, retorna a neutral y confirma por MQTT, sin bloquear el hilo del broker.

El *callback* de MQTT del nodo Motor interpreta los tópicos suscritos y solo dispara cuando el sistema está activo, como muestra el Código 2[cite: 290].

```
1 if (strcmp(topic, TOPIC_COMANDO) == 0) {
2     if (strcmp(data, "iniciar") == 0) {
3         sistema_activo = true;
4         motor_avanzar(MOTOR_VEL_NORMAL);
5         esp_mqtt_client_publish(client, TOPIC_MOTOR, "activo", 0, 1, 0);
6     } else if (strcmp(data, "detener") == 0) {
7         sistema_activo = false;
8         motor_detener();
9         esp_mqtt_client_publish(client, TOPIC_MOTOR, "detenido", 0, 1, 0);
10    }
11 }
12 else if (strcmp(topic, TOPIC_FRUTA) == 0 && sistema_activo) {
13     if (strcmp(data, "rechazada") == 0)
14         xTaskNotifyGive(tarea_servos_handle); // NOTIFICA a la tarea, no bloquea
15 }
```

Listing 2: Manejo de eventos MQTT en el nodo Motor (caso de datos recibidos con servomotores).

9.5. Máquinas de estados finitos (FSM) actualizadas

El comportamiento de cada nodo se modela como una máquina de estados finitos[cite: 294]. La Figura 3 presenta la FSM del nodo Motor actualizada con servomotores: tras el arranque conecta WiFi y MQTT, y alterna entre **DETENIDO** y **ACTIVO** según los comandos; en **ACTIVO**, un veredicto **rechazada** dispara los servomotores[cite: 295, 296].

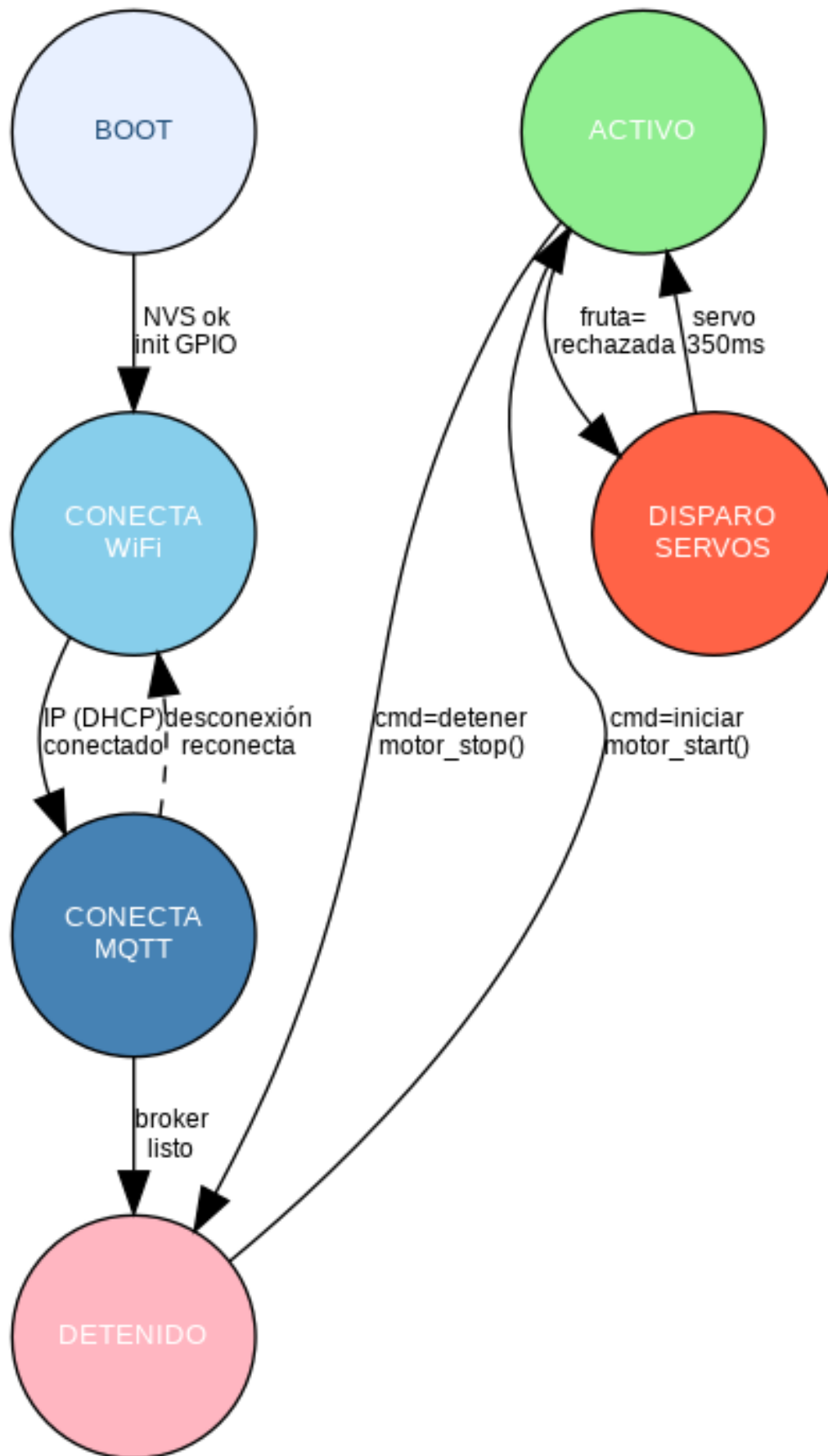


Figura 3: FSM del nodo ESP32 Motor actualizada con servomotores MG996R. El estado DISPARO SERVOS se ejecuta en una tarea FreeRTOS separada (no bloqueante, duración [1, 250...][1, 200-207]

La Figura 4 muestra la FSM de navegación de pantallas de la consola: el botón PANTALLA rota entre CONTADORES, DIAGNÓSTICO y ALERTA [cite: 297]; tres rechazos consecutivos fuerzan el salto a ALERTA y lo muestran en pantalla OLED [cite: 298]; el botón RESET limpia contadores y alerta[cite: 299].

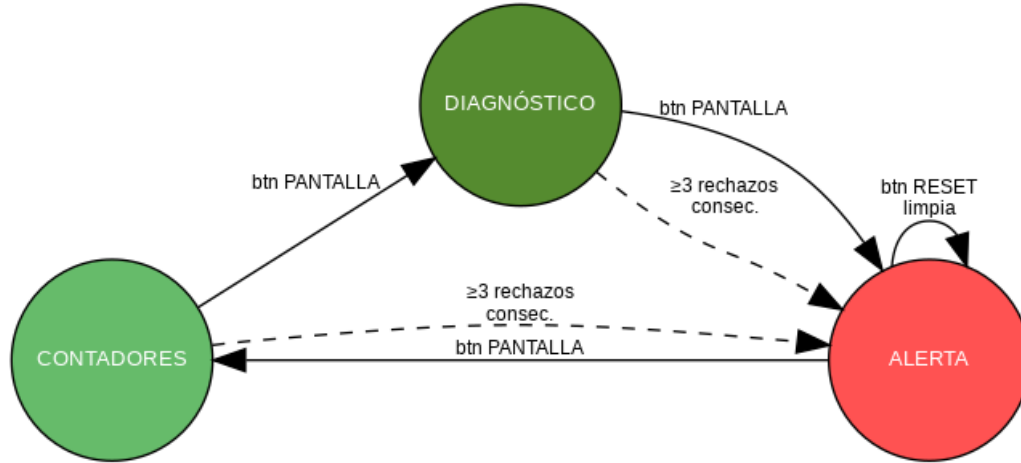


Figura 4: FSM de pantallas de la consola OLED. La transición forzada a ALERTA se dispara con tres rechazos consecutivos[cite: 299, 300].

10. Caracterización y gráficas de respuesta

10.1. Señal PWM del motor

El periférico LEDC genera la señal PWM a 10 kHz sobre RPWM (GPIO25)[cite: 300]. Con MOTOR_VEL_NORMAL=150 sobre 255, el ciclo de trabajo es del 58.8 %, como se observa en la Figura 5[cite: 301]. La Figura 6 muestra la relación lineal entre el valor de *duty* (8 bits) y la tensión media aplicada al motor, y el punto de operación elegido[cite: 302].

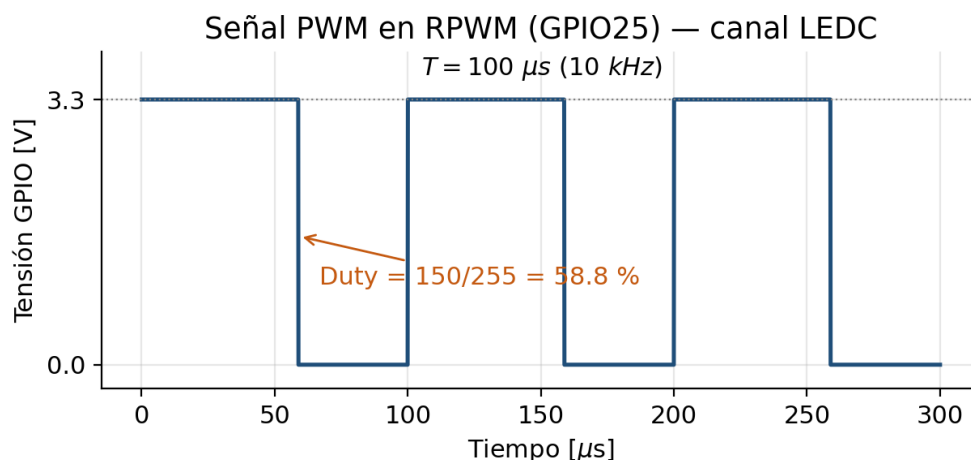


Figura 5: Señal PWM a 10 kHz en GPIO25 con ciclo de trabajo de 150/255 (58.8 %). Período de 100 μs [cite: 303, 304].

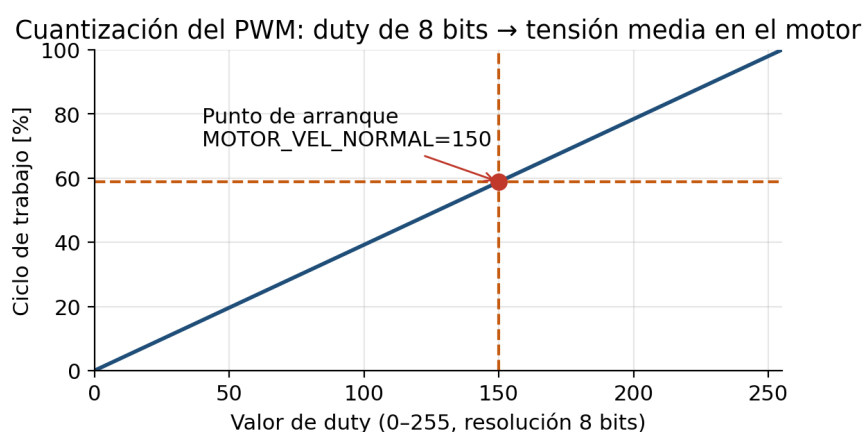


Figura 6: Cuantización del PWM: el *duty* de 8 bits se mapea linealmente a la tensión media en el motor. En rojo, el punto de arranque (*duty*=150) [cite: 304, 305].

10.2. Cronograma del disparo de servomotores

La Figura 7 muestra la temporización del disparo no bloqueante: el evento MQTT rechazada provoca el giro de los servomotores (GPIO4, GPIO5) desde 90° a 180° durante `SERVO_MS=300 ms`; los servos actúan mecánicamente y, al retornar a neutral, el nodo publica `sistema/servos=expulsado` [cite: 308].

Cronograma: Disparo de Servomotores MG996R
0°=neutral, 180°=expulsión, 350ms ciclo total

TIEMPO (ms) →								
0	50	100	200	250	300	350	400	500
EVENTO: MQTT fruta=rechazada								
Servo 1 & 2: 0° → 180° (EXPULSA)								
Neutral (90°)			Expulsión (180°)				Retorno (0°)	
GPIO27 (relé): LOW → HIGH → LOW								

Figura 7: Cronograma del disparo de servomotores gestionado por la tarea FreeRTOS dedicada. Incluye rampas de giro y estabilización[cite: 309, 310].

10.3. Presupuesto de latencia extremo a extremo

La Figura 8 descompone el tiempo total desde la captura hasta la expulsión[cite: 310]. La etapa dominante es la inferencia de visión en la PC [cite: 311]; las etapas de red y reacción embebida suman pocas decenas de milisegundos, de modo que el total se mantiene holgadamente por debajo de los 2s (RNF-01) y determina la velocidad máxima admisible de la banda[cite: 312].

Presupuesto de latencia extremo a extremo (total ≈ 1548 ms < 2 s, RNF-01)

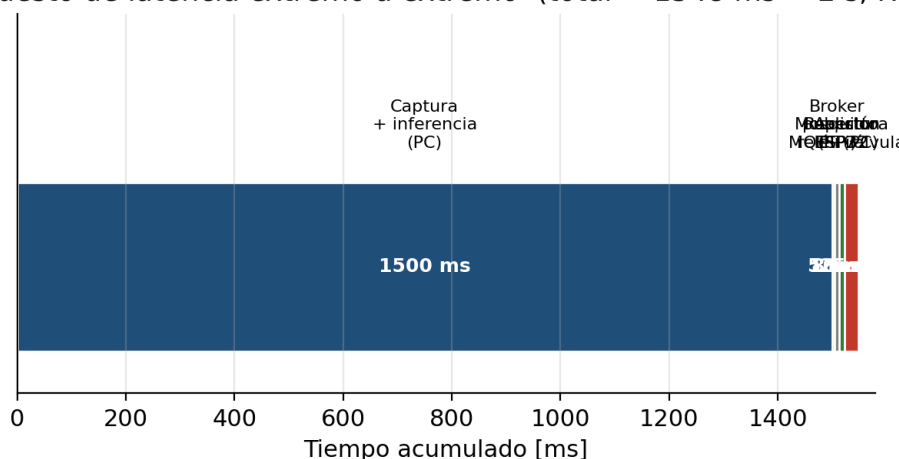


Figura 8: Presupuesto de latencia extremo a extremo (valores típicos/de especificación). La inferencia domina; la cadena MQTT + reacción embebida es despreciable frente a ella[cite: 313, 314].

11. Aplicación real: producción de tomate en Tarija

El fin último de la cinta es servir como herramienta de control de calidad poscosecha accesible a pequeños productores de tomate[cite: 314]. Tarija reúne condiciones idóneas: pese a ser el departamento más pequeño de Bolivia, dispone de unas 189.268 hectáreas con vocación productiva agrícola El Diario, 2026, y el tomate es uno de los cultivos relevantes de sus valles El País (Tarija), 2023[cite: 315].

Comunidad propuesta: Chocloca (municipio de Uriondo, Valle Central). Se propone como caso de aplicación la comunidad de **Chocloca**, uno de los nueve distritos del municipio de Uriondo (provincia José María Avilés), a unos 25 km al sur de la ciudad de Tarija, en el Valle Central[cite: 316]. El municipio agrupa 53 comunidades campesinas, de las cuales más del 90 % se dedica a la agricultura, con el tomate entre sus principales cultivos junto a la vid, la papa y el morrón El País (Tarija), 2023[cite: 317]. Chocloca pertenece a la cuenca del río Camacho, de clima templado (temperatura media diaria máxima cercana a 26 °C y precipitación media anual en torno a 740 mm) Erazo Campos, 2003; Wikipedia, 2024, lo que favorece varias campañas hortícolas al año[cite: 318].

Por qué es un caso idóneo. Varios factores hacen pertinente el prototipo en esta comunidad:

- **Clasificación manual y pérdidas.** La selección del tomate se hace a mano [cite: 319]; un fruto deteriorado no detectado contamina el resto del lote, generando pérdidas[cite: 320].

- **Vulnerabilidad de la oferta.** La producción de tomate en Tarija llegó a caer hasta un 60 % por sequías y heladas [cite: 321]; en un contexto de oferta ajustada, reducir el descarte tiene impacto económico directo[cite: 322].
- **Contexto rural sin Internet.** La arquitectura del proyecto opera sobre una red WiFi local autónoma (RPi como Access Point) y una consola a batería [cite: 323]; no requiere conectividad externa (RNF-05)[cite: 324].

Referencias Bibliográficas

- Banks, A., & Gupta, R. (2014). *MQTT Version 3.1.1* (OASIS Standard). OASIS. Consultado el 6 de junio de 2026, desde <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>
- Barry, R. (2016). *Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide*. Real Time Engineers Ltd.
- El Diario. (2026). *Tarija tiene más de 1,6 millones de hectáreas con alto potencial productivo agropecuario*. Consultado el 7 de junio de 2026, desde <https://www.eldiario.net/portal/2026/04/16/tarija-tiene-mas-de-16-millones-de-hectareas-con-alto-potencial-productivo-agropecuario/>
- El País (Tarija). (2023). *Avilés y la variedad de su producción agrícola*. Consultado el 7 de junio de 2026, desde https://elpais.bo/tarija/20230415_aviles-y-la-variedad-de-su-produccion-agricola.html
- Erazo Campos, O. (2003). Uso de la totora en la producción agrícola de la cuenca del río Camacho, Tarija, Bolivia. *LEISA Revista de Agroecología*, 19(3).
- Espressif Systems. (2023). *ESP32 Technical Reference Manual* (Version 5.1). Espressif Systems. Consultado el 6 de junio de 2026, desde https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- Espressif Systems. (2024). *ESP-IDF Programming Guide (v5.x): Wi-Fi, MQTT, LEDC y FreeRTOS*. Consultado el 6 de junio de 2026, desde <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/>
- Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing* (4th). Pearson.
- IEEE. (2021). *IEEE Standard for Information Technology – Telecommunications and Information Exchange between Systems – Local and Metropolitan Area Networks (IEEE 802.11-2020)* (Standard). Institute of Electrical y Electronics Engineers.
- Infineon Technologies. (2004). *BTS7960 High Current PN Half Bridge – Data Sheet*. Infineon Technologies AG. Consultado el 6 de junio de 2026, desde <https://www.infineon.com/dgdl/bts7960.pdf>
- Light, R. A. (2017). Mosquitto: server and client implementation of the MQTT protocol. *The Journal of Open Source Software*, 2(13), 265. <https://doi.org/10.21105/joss.00265>
- OpenCV Development Team. (2024). *OpenCV Documentation*. Consultado el 1 de abril de 2026, desde <https://docs.opencv.org>
- Universidad Católica Boliviana "San Pablo". (2026). *Guía de proyectos del Departamento de Ciencias de la Tecnología e Innovación (DCT)*. Consultado el 30 de marzo de 2026, desde <https://www.ucb.edu.bo>
- Wikipedia. (2024). *Uriondo (datos climáticos de Chocloca, 1975–2010)*. Consultado el 7 de junio de 2026, desde <https://en.wikipedia.org/wiki/Uriondo>